

Amortized Predictability-aware Training Framework for Time Series Forecasting and Classification

Xu Zhang
Shanghai Key Laboratory
of Data Science, College of
Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
xuzhang22@m.fudan.edu.cn

Peng Wang*
Shanghai Key Laboratory
of Data Science, College of
Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
pengwang5@fudan.edu.cn

Yichen Li
Department of Electrical
and Computer Engineering
University of British
Columbia (UBC)
Vancouver, Canada
yli262@student.ubc.ca

Wei Wang
Shanghai Key Laboratory
of Data Science, College of
Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
weiwang1@fudan.edu.cn

Abstract

Time series data are prone to noise in various domains, and training samples may contain low-predictability patterns that deviate from the normal data distribution, leading to training instability or convergence to poor local minima. Therefore, mitigating the adverse effects of low-predictability samples is crucial for time series analysis tasks such as time series forecasting (TSF) and time series classification (TSC). While many deep learning models have achieved promising performance, few consider how to identify and penalize low-predictability samples to improve model performance from the training perspective. To fill this gap, we propose a general Amortized Predictability-aware Training Framework (APTF) for both TSF and TSC. APTF introduces two key designs that enable the model to focus on high-predictability samples while still learning appropriately from low-predictability ones: (i) a Hierarchical Predictability-aware Loss (HPL) that dynamically identifies low-predictability samples and progressively expands their loss penalty as training evolves, and (ii) an amortization model that mitigates predictability estimation errors caused by model bias, further enhancing HPL's effectiveness. The code is available at <https://github.com/Meteor-Stars/APTF>.

CCS Concepts

• Information systems → Spatial-temporal systems; • Computing methodologies → Machine learning.

Keywords

time series forecasting, time series classification, time series analysis, deep learning techniques, noise robust learning

ACM Reference Format:

Xu Zhang, Peng Wang, Yichen Li, and Wei Wang. 2026. Amortized Predictability-aware Training Framework for Time Series Forecasting and Classification. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792637>

*Peng Wang is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792637>

1 Introduction

Time series (TS) data is ubiquitous in web applications, such as ECG signals in EHRs [29], fund sales on Alipay [41], and IoT sensor data [43]. By analyzing temporal patterns, time series forecasting (TSF) and time series classification (TSC) can help make decisions, e.g., predict future trends for inventory planning [42] or identify events occurring within the current temporal patterns for health-care monitoring [12]. As a result, these techniques have become increasingly important in modern web technologies [15, 43].

TS data are prone to noise. For instance, industrial sensors may suffer recording errors due to wear and environmental factors [26]; ECG signals can be corrupted by electromyographic interference in medical applications [27]; and stock sequences in the financial sector often exhibit noise caused by market fluctuations [11]. As a result, TS samples frequently contain low-predictability patterns [9], which deviate from the normal data distribution and exhibit higher noise levels. Moreover, deep neural networks have been demonstrated to have a high capacity to fit noisy data during training, which will further result in poor performance and generalization on the test data [40]. Importantly, this phenomenon does not change with the choice of training optimizations or network architectures [18, 40]. **The high loss values of low-predictability samples can dominate gradient updates, causing training oscillations and trapping the model in poor local minima [9].** Therefore, mitigating the adverse impact of low-predictability samples is essential for improving model convergence.

Although deep-learning-based methods perform well in TSF [20, 36] and TSC [7, 32], they typically treat all training samples equally and ignore how to identify and handle low-predictability samples to achieve better convergence. Ideally, such samples should incur smaller losses, enabling the model to learn from them without overemphasis while paying more attention to high-predictability samples. In the computer vision domain, Co-teaching [14] removes noisy samples based on loss values and shows strong robustness to label noise. This leads to the question: *Can Co-teaching be adapted to time series tasks, and what modifications are required?*

We show that adopting Co-teaching for time series is non-trivial, as directly applying the vanilla Co-teaching framework may not improve TS models and can even degrade their performance. Co-teaching is designed for settings with extremely noisy labels, where such samples are explicitly harmful and can be safely discarded. In contrast, our setting does not involve extreme noise. **Low predictability samples may still provide useful information,**

and discarding them can waste knowledge and harm performance. The key challenge is to learn from these samples while reducing the risk of overfitting their noise. To this end, we propose an Amortized Predictability-aware Training Framework (APTF) for TSF and TSC tasks.

Noisy and low-predictability samples both deviate from the data distribution. Given Co-teaching's great success in reducing noise by discarding samples with high-loss values, we similarly use high-loss values to identify low-predictability samples. However, deep models have been proven to first memorize clean (high-predictability) samples, and gradually adapt to noisy (low-predictability) samples as training proceeds [1]. This suggests that loss values can reliably reflect sample predictability in early training, but become unreliable as training progresses due to overfitting, which is a phenomenon we term the *predictability evolution* issue.

This motivates us to propose a Predictability-aware Training Framework (PTF) that dynamically identifies low-predictability samples as training evolves. The core of PTF is the Hierarchical Predictability-aware Loss (HPL). It groups samples into multiple buckets based on their predictability from high to low (determined by loss magnitude), and penalizes the gradients of low-predictability samples by assigning progressively smaller loss weights. Then, the hierarchical bucketing strategy across training stages further addresses the *predictability evolution* issue. As a result, HPL emphasizes highly predictable samples while down-weighting low-predictability ones, reducing the risk of overfitting to noise while still learning appropriately from low-predictability ones.

In PTF, the process of finding low-predictability samples is dynamically evolving by a single model itself.

However, the model's decisions can be overconfident and biased [2, 13], leading to predictability estimation errors where highly predictable samples are misestimated as low-predictability ones, vice versa. When using only a single model, accumulated estimation errors may cause over-penalization of highly predictable samples and degrade performance. To alleviate this, we introduce an amortization model to reduce predictability estimation error and further improve PTF, resulting in the Amortized Predictability-aware Training Framework (APTF). Our contributions are as follows:

- We propose a general Amortized Predictability-aware Training Framework (APTF) for TSF and TSC tasks. To our knowledge, this is the first framework that improves training for both tasks by dynamically identifying and penalizing low-predictability samples throughout training.
- In APTF, we propose a Hierarchical Predictability-aware Loss (HPL) to address the *predictability evolution* issue. HPL enables the model to focus on high-predictability samples while still learning appropriately from low-predictability ones, with a reduced risk of overfitting to their noise. An amortization model is further designed to alleviate predictability estimation errors, improving the effectiveness of HPL.
- We conduct extensive experiments on 11 TSF datasets and 128 TSC datasets from the UCR archive. Results show that APTF improves convergence accuracy and generalization across eleven TSF and five TSC models, reducing prediction errors by an average of 2%–9.79% on long-term and 3%–15.36% on short-term TSF tasks.

2 Related work

Time series forecasting (TSF). Initially, Recurrent Neural Networks (RNNs) [8, 19] were popular for TSF, but their recurrent structure limits scalability. Temporal Convolutional Networks (TCNs) [30], based on causal and dilated convolutions, improve parallelism and are widely used in TSF [30]. However, both RNNs and CNNs struggle to capture long-term dependencies due to structural or receptive-field limitations. To address this, many methods adapt Transformers [34] to model both short- and long-term dependencies, such as Autoformer [38], Scaleformer [31], and MLF [42]. Recently, MLP-based linear models have also shown strong performance [6, 10, 46], including NLinear [39], N-HITS [5], and LSINet [44]. In addition, CNN-based models [24] and cross-attention methods [20] achieve a favorable balance between efficiency and accuracy.

Time series classification (TSC). Recently, deep learning has gained popularity in TSC due to its strong feature extraction ability, high accuracy, and scalability [16]. One-dimensional CNNs (1D-CNNs) are widely used for TSC because of their effectiveness in capturing local temporal features [33], leading to methods such as InceptionTime [17], OS-CNN [32], FCN, and ResNet [37]. More recently, FormerTime [7] integrates 1D-CNNs with Transformers to jointly model local and global time-series features for TSC.

Methods for predictability-aware learning. Some studies mitigate the impact of low-predictability samples by modifying loss computation. For example, Co-teaching [14] discards high-loss samples to handle extremely noisy labels, thereby enhancing the model's robustness. Nevertheless, such samples can still be meaningful in our setting, motivating the proposed Hierarchical Predictability-aware Loss (HPL). Self-paced learning [21] instead learns from low-loss samples first and gradually includes harder ones, while adaptive robust loss [3] suppresses outliers via learnable loss parameters. In TSF, WaveBound [9] avoids overfitting unpredictable steps by adjusting gradients when prediction errors are lower than estimated bounds.

3 Amortized predictability-aware training framework (APTF)

APTF is illustrated in Figure 1(b). Its core component is the Hierarchical Predictability-aware Loss (HPL), detailed in Algorithm 2 and Figure 2. HPL adopts a hierarchical bucketing strategy to penalize the loss of low-predictability samples as training evolves, allowing the model to learn from them while reducing the risk of overfitting their noise. By introducing an amortization model, predictability estimation bias is first absorbed and then passed to the source model with a one-step delay (red arrows in Figure 1(b)), mitigating the misestimation of sample predictability caused by overconfidence. Notably, HPL alone already yields substantial performance gains, while the amortization model further enhances its effectiveness.

3.1 Hierarchical predictability-aware loss (HPL)

3.1.1 Low-predictability to high-predictability buckets . We define the training set as $\mathcal{D} = \{(X_1, Y_1), \dots, (X_N, Y_N)\}$, in which $X_N = [x_1, x_2, \dots, x_t] \in \mathbb{R}^{t \times v}$ with length t for all v time variables. Y_N varies depending on the downstream task. For example, in the forecasting task, $Y_N = [x_{t+1}, x_{t+2}, \dots, x_{t+m}]$ are the consecutive time values of X_i , while in the classification task, $Y_N \in 1, \dots, C$ are

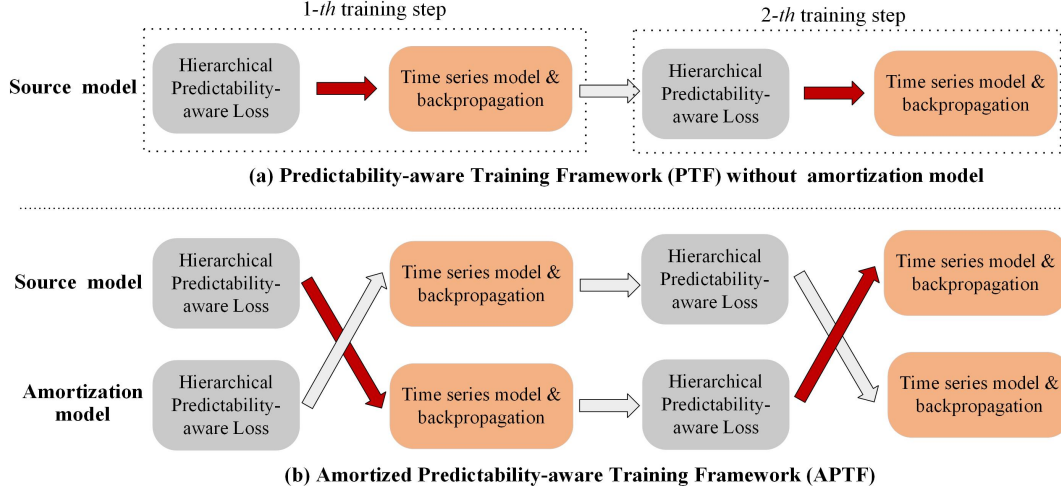


Figure 1: The proposed general training framework APTF for time series forecasting and classification.

C class labels. During training, we optimize the TS model $f(X, \Theta)$ through the basic function:

$$\arg \min_{\Theta} \sum_{(X_i, Y_i) \in \mathcal{D}} \mathcal{L}(f(X_i, \Theta), Y_i) \quad (1)$$

where $\mathcal{L}$ denotes Mean Squared Error (MSE) Loss for forecasting or cross-entropy loss for classification. For clarity, we illustrate the following method using the forecasting task as an example.

Low-predictability samples typically incur larger losses, which can cause training oscillations and convergence to poor local minima. Thus, it is necessary to identify these samples and regularize them to prevent the model from overfocusing on them and avoid harming the model's learning capacity from high-predictability samples. Since loss values are effective indicators of noisy samples [14], we use high-loss values to identify low-predictability samples in our setting.

We first uniformly divide samples into K buckets $B_1, \dots, B_K$ according to ascending loss values. **Buckets with larger indices contain samples with higher loss and thus lower predictability**, e.g., B_K contains the least predictable samples. Each bucket has $N_k^B = \lfloor \frac{N}{K} \rfloor$ samples, where N is the batch size and $\lfloor \cdot \rfloor$ denotes the floor operation. After filling the first $K - 1$ buckets evenly, all remaining samples are assigned to the last bucket. As shown in Figure 2(a), when the samples can be evenly divided, $Len(B_1) = Len(B_2) = \dots = Len(B_K)$, where $Len(\cdot)$ denotes the number of samples in a bucket. The example is shown in Figure 2(a), when N is divisible by K , $Len(B_1) = \dots = Len(B_K)$, where $Len(\cdot)$ denotes the number of samples in a bucket.

3.1.2 Loss weights for paying more attention to high predictability samples. The basic buckets $\{B_1, \dots, B_K\}$ group samples by predictability. To reduce the adverse influence of low-predictability samples, we assign smaller loss weights to higher-index buckets and larger weights to lower ones, i.e., $\{W_1 > W_2 > \dots > W_K\}$. This amplifies the gradients of high-predictability samples, guiding the model to focus more on them. We empirically observe that uniformly decreasing weights yield good performance in practice. The

Algorithm 1: Basic *Predictability_Aware_Loss*.

Input:

- (1) K buckets $\{B_1, B_2, \dots, B_K\}$, each bucket containing the loss value of samples with different predictability levels.
- (2) K weights $\{W_1, W_2, \dots, W_K\}$ for penalizing the loss values in different buckets.
- (3) Initialized final loss value $\mathcal{L} = 0$.

Output: The loss value $\mathcal{L}$.

```

1 for  $j = 1$  to  $K$  do
2    $\mathcal{L}_j = \text{Average}(B_j \cdot W_j)$ 
3    $\mathcal{L} = \mathcal{L} + \mathcal{L}_j$ 
4 end
5 return  $\mathcal{L}$ 

```

resulting objective is termed the basic predictability-aware loss, as shown in Algorithm 1 and Figure 2(a).

3.1.3 Predictability-aware loss with predictability evolution.

The basic predictability-aware loss in Algorithm 1 adopts a fixed number of buckets throughout training, resulting in a fixed gradient penalty range for low-predictability samples. It fails to account for the adverse effects of overfitting during training, which may lead to the low-predictability sample not being penalized because it was assigned to a lower-index bucket due to its lower loss value resulting from overfitting. This phenomenon, referred to as the *predictability evolution* issue, indicates that loss values effectively estimate sample predictability in early training stages but become less accurate as training progresses.

To address this, we first uniformly divide the training into S stages at the training epoch interval ε to dynamically reduce the number of buckets as the stage increases. The next training stage will be activated when the indicator $tag = 1$:

$$tag = \begin{cases} 1 & \text{if current epoch \% } \varepsilon = 0 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The decreased number of buckets at different training stages is illustrated in Figure 2(b), which shows an example of three training stages. As training progresses, samples originally in high-weight

Algorithm 2: Amortized hierarchical predictability-aware loss with S training stages**Input:**

- (1) Hierarchical G bucket groups $\{\mathcal{B}_1, \dots, \mathcal{B}_G\}$ and weight groups $\{\mathcal{W}_1, \dots, \mathcal{W}_G\}$ from **source model** at s -th training stage.
- (2) G bucket groups $\{\mathcal{B}_1, \dots, \mathcal{B}_G\}$ and weight groups $\{\mathcal{W}_1, \dots, \mathcal{W}_G\}$ from **amortization model** at s -th training stage.
- (3) Initialized loss value $\mathcal{L} = 0$ and $\tilde{\mathcal{L}} = 0$ for source model and amortization models.
- (4) Basic function *Predictability_Aware_Loss* ($\cdot$) in Algorithm 1
- (5) Whether to use amortization model: *amortization*=True.

Output:

Loss $\mathcal{L}$ and $\tilde{\mathcal{L}}$ for optimizing source and amortization model.

```

1 for  $i = 1$  to  $S$  do
2   if  $i=1$  then
3     if amortization=True then
4        $\mathcal{L} = \mathcal{L} + \text{Predictability\_Aware\_Loss}(\mathcal{B}_1, \mathcal{W}_1)$ 
5        $\tilde{\mathcal{L}} = \tilde{\mathcal{L}} + \text{Predictability\_Aware\_Loss}(\mathcal{B}_1, \mathcal{W}_1)$ 
6     else
7        $\mathcal{L} = \mathcal{L} + \text{Predictability\_Aware\_Loss}(\mathcal{B}_1, \mathcal{W}_1)$ 
8        $\tilde{\mathcal{L}} = 0$ 
9     end
10    return  $\mathcal{L}, \tilde{\mathcal{L}}$ 
11  else
12    for  $j = 1$  to  $i$  do
13      if amortization=True then
14         $\mathcal{L} = \mathcal{L} + \text{Predictability\_Aware\_Loss}(\mathcal{B}_j, \mathcal{W}_j)$ 
15         $\tilde{\mathcal{L}} = \tilde{\mathcal{L}} + \text{Predictability\_Aware\_Loss}(\mathcal{B}_j, \mathcal{W}_j)$ 
16      else
17         $\mathcal{L} = \mathcal{L} + \text{Predictability\_Aware\_Loss}(\mathcal{B}_j, \mathcal{W}_j)$ 
18         $\tilde{\mathcal{L}} = 0$ 
19      end
20    end
21    return  $\mathcal{L}/G, \tilde{\mathcal{L}}/G$ 
22  end
23 end

```

buckets are gradually reassigned to lower-weight ones by reducing the total number of buckets and removing the highest loss weights. This progressively enlarges the range of penalized low-predictability samples. The updated buckets and weights are then fed into Algorithm 1 to compute the training loss for optimization.

3.1.4 Hierarchical Predictability-aware Loss with predictability evolution. Although the design in Figure 2(b) mitigates the *predictability evolution* issue, it still has some limitations. **First**, the loss weights used to penalize samples vary across different training stages, which can lead to significant changes in the model's gradient direction, potentially causing unstable training or falling into poor local optima. **Second, as the training stage evolves, more samples will be penalized with lower weights and the gradient strength becomes weaker, which can prevent some highly predictable samples from obtaining sufficient gradients to learn due to their low loss values**, thereby harming model performance.

To address these limitations, we further propose the **Hierarchical Predictability-aware Loss (HPL)**, illustrated in Figure 2(c). We define a *bucket group* as $\mathcal{B}_1, \dots, \mathcal{B}_G$, where each group contains several buckets, e.g., $\mathcal{B}_1 = \{B_1, \dots, B_K\}$, with corresponding weight groups $\mathcal{W}_1, \dots, \mathcal{W}_G$. As training progresses, the number of bucket groups gradually increases, while newly added groups contain fewer buckets, leading to more samples per bucket. Meanwhile,

lower weights are assigned to progressively expand the penalty on low-predictability samples, following the same principle as before. The “hierarchy” indicates that at the S -th training stage ($S > 1$), the number of bucket groups is always greater than one. As shown in Figure 2(c), **stage 1** contains a single bucket group with 5 buckets, whereas **stage 2** contains two bucket groups with 5 and 3 buckets, respectively. In contrast, without HPL, each training stage has only one bucket group (Figure 2(b)).

As training progresses, we incorporate previous bucket partition strategies into the loss computation and average the losses across different bucket groups. **On one hand**, this avoids abrupt gradient changes that could cause training oscillations or poor local optima. **On the other hand**, it compensates the loss of highly predictable samples, thereby amplifying their gradients to mitigate the risks of harming the model's learning ability due to over-penalization. This achieves a balance where high-predictability samples are sufficiently fitted while low-predictability ones are appropriately penalized. For S training stages, the Hierarchical Predictability-aware Loss is computed as shown in Algorithm 2 (with *amortization* set to False).

3.2 Amortized hierarchical predictability-aware loss

In HPL, low-predictability samples are dynamically identified by a single model, which may introduce bias due to overconfidence and lead to misidentification of sample predictability [2, 13]. Inspired by co-training [4], we introduce an amortization model to reduce predictability estimation error caused by model bias. By mutually estimating sample predictability, the bias of the source model is first amortized (absorbed) by the amortization model and then propagated back after a one-step delay, as shown by the red arrows in Figure 1(b). This reduces the risk of over-penalizing highly predictable samples due to misidentification and further improves HPL performance. The pseudocode of the amortized hierarchical predictability-aware loss is provided in Algorithm 2.

4 Experiments and results

4.1 Experimental settings

Datasets for short-term TSF. We introduce fund sales datasets from a financial competition¹ to validate the effectiveness of our method. **The reason is that fund sales exhibit significant volatility under the influence of complex factors such as policy, public opinion, and market trends, making them more prone to low-predictability patterns.** They are divided into three groups based on the holding period for comprehensive experiment evaluation, called Fund1 (containing 66 fund sales datasets), Fund2 (107 fund sales datasets), and Fund3 (133 fund sales datasets). Details can be found in Appendix Section A.1.2 and Figure 5.

Datasets for long-term TSF. We evaluate the performance of our proposed framework on eight popular datasets, including Exchange, Weather, Electricity, Traffic, and four ETT datasets (ETTh1, ETTh2, ETTm1, ETTm2). These datasets involve applications in industrial machines, energy, and weather domains. They have been widely employed in the literature for multivariate forecasting

¹<https://tianchi.aliyun.com/competition/entrance/532093/information>

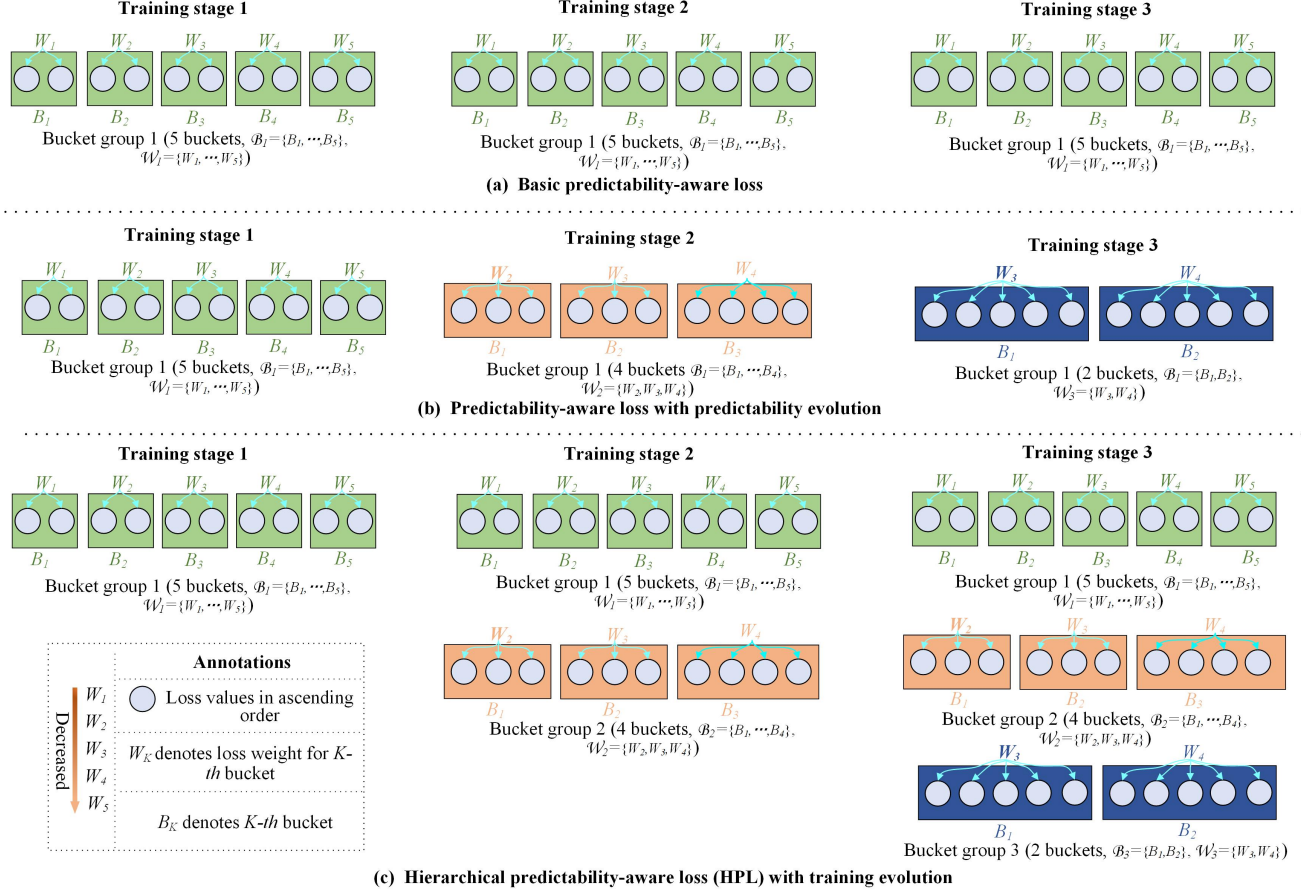


Figure 2: Illustration of different bucketing strategies for computing the predictability-aware loss. (a) A fixed number of buckets is used throughout training, ignoring the *predictability evolution* issue. (b) The number of buckets decreases as training progresses, addressing *predictability evolution* but discarding previous bucket partitions. (c) Hierarchical buckets consider both *predictability evolution* and previous bucket partitioning strategies.

tasks [25, 35, 38]. Following previous works, each dataset is split into training, validation, and testing sets in a 7:1:2 ratio. In short-term forecasting, we merged the training, validation, and test sets across all fund datasets for model training and evaluation because they share similar TS patterns.

Datasets for TSC. We validate the effectiveness of the proposed framework on 128 univariate TSC datasets from the UCR archive², which are widely used for evaluating TSC methods [17, 32, 37]. In these original datasets, the training and testing set have been well processed. We do not take any processing for these datasets for a fair comparison.

Baselines For TSF. We use eleven advanced baselines to extensively evaluate our methods, including **one CNN-based** model, ModernTCN [24], seven transformer-based models, CATS [20], TimeXer [36], PatchTST [25], Autoformer [38], Non-stationary Transformer (NSformer) [23], Scaleformer [31], and Informer [45]. We also include three linear models: TimeMixer, NLinear [39], and

N-HiTS [5]. **For TSC**, five advanced baselines are selected for evaluating our training framework APTF, including InceptionTime [17], OS-CNN [32], FCN, ResNet [37] and FormerTime [7].

Evaluation metrics. Following the previous works, we use Mean Squared Error (MSE) and Mean Absolute Error (MAE) metrics for TSF methods evaluation [25, 31, 38] while accuracy for TSC evaluation [7, 17, 32]. Besides, the competition evaluation metric WMAPE (Weighted Mean Absolute Percentage Error) is also used to assess prediction accuracy, which measures the accuracy of predictions on larger transactions.

Implementation details. When APTF is combined with baseline models, all experimental settings (e.g., model hyperparameters and optimizer) are consistent for fair comparison. More details are shown in Appendix A.1.1 due to limited space. For each experiment, we independently run four times with four different seeds, and the average metrics and standard deviations are reported. Experiments are conducted on NVIDIA GeForce RTX 3090 GPU on PyTorch.

Due to limitations in both the main text and the Appendix, some results could not be fully presented, including (i) full results and standard deviations on each prediction length in short-term

²<https://www.timeseriesclassification.com/>

Table 1: Short-term TSF results of APTF for forecasting 1, 5, 8, and 10 future steps, where the average WMAPE over all horizons is reported. APTF also shows improvements over baselines CATS, TimeXer, and ModernTCN, as shown in Appendix Table 9.

	TimeMixer		PatchTST		Autoformer		NSformer		Scaleformer		NLinear		N-HiTS		Informer	
	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original
Fund1	87.914	89.963	86.549	88.811	103.286	110.295	95.112	97.198	100.027	107.31	102.536	106.636	86.856	91.69	102.912	106.952
Fund2	84.356	85.512	85.094	86.472	99.712	107.141	91.516	94.406	96.736	109.01	90.791	93.538	86.13	91.531	91.407	97.193
Fund3	83.659	85.14	84.821	86.593	98.838	107.063	90.364	92.646	95.235	107.405	90.542	93.652	84.719	90.332	90.476	96.589

Table 2: Long-term forecasting results for horizons of 96, 192, 336, and 720, reporting averaged MSE and MAE. Due to space constraints, partial results for each horizon are provided in Appendix 13. APTF also shows improvements over baselines CATS, TimeXer, and ModernTCN, as shown in Appendix Table 10 and Table 12.

	TimeMixer		PatchTST		Autoformer		NSformer		Scaleformer		NLinear		N-HiTS		Informer	
	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original
ETTh1	0.492	0.517	0.40	0.43	0.492	0.504	0.511	0.523	0.543	0.55	0.453	0.469	0.491	0.516	1.038	1.058
ETTh2	0.395	0.415	0.366	0.37	0.398	0.434	0.407	0.434	0.409	0.441	0.38	0.383	0.546	0.599	1.577	2.357
ETThm1	0.398	0.407	0.357	0.369	0.506	0.556	0.553	0.599	0.506	0.518	0.375	0.377	0.385	0.399	0.675	0.74
ETThm2	0.3	0.304	0.278	0.29	0.351	0.36	0.337	0.356	0.354	0.414	0.286	0.291	0.311	0.334	1.55	2.092
weather	0.264	0.269	0.241	0.244	0.383	0.386	0.307	0.314	0.452	0.534	0.263	0.27	0.257	0.262	0.508	0.595
Exchange	0.371	0.381	0.367	0.374	0.455	0.488	0.444	0.46	0.492	0.562	0.37	0.376	0.356	0.402	0.875	0.924
Electricity	0.288	0.306	0.25	0.252	0.335	0.344	0.321	0.33	0.336	0.35	0.319	0.355	0.255	0.262	0.737	0.755
Traffic	0.613	0.648	0.498	0.501	0.618	0.642	0.633	0.661	0.639	0.665	0.7	0.796	0.489	0.503	0.953	0.976

(corresponding to Table 1 and Table 9) and long-term TSF tasks (corresponding to Table 2 and Table 10), and (ii) full results and standard deviations on each time series classification dataset (corresponding to Table 3). We will show them in the full version paper.

4.2 Results of time series forecasting (TSF) and time series classification (TSC)

Low-predictability samples often yield large losses, causing training oscillations and poor local convergence. **APTF mitigates this by maintaining normal learning on high-predictability samples while penalizing losses from low-predictability ones.** This enables the model to learn useful patterns without overfitting noise, improving performance in both TSF and TSC tasks. Due to space limitations, the results of short-term and long-term tasks for baselines CATS, TimeXer, and ModernTCN are presented in Appendix Table 9 and Table 10.

4.2.1 Short-term forecasting. Results are presented in Table 1. Across various prediction steps, our training framework APTF consistently improves the accuracy of eight baseline models, including five transformer-based models and three linear models. Specifically, the average improvements for the transformer models are 2.06%, 6.99%, 2.56%, 9.79%, and 5.35%, corresponding to models PatchTST, Autoformer, NSformer, Scaleformer, and Informer, respectively. The average improvements for the linear models are 1.79%, 3.37%, and 5.79%, corresponding to TimeMixer, NLinear, and N-HiTS. The maximum improvements are more obvious.

4.2.2 Long-term forecasting. Results in Table 2 also show that our training framework APTF consistently improves the accuracy of five transformer-based models and three linear models on different forecasting steps. Specifically, the average improvements for the transformer models are 2.01%, 4.78%, 4.5%, 8.15%, and 13.14%, corresponding to models PatchTST, Autoformer, NSformer, Scaleformer, and Informer, respectively. The average improvements for the linear models are 3.3%, 3.24%, and 5.75%, corresponding to TimeMixer,

NLinear, and N-HiTS. The maximum improvement ratios for them are more significant.

Table 3: TSC task. The average classification accuracy over 128 UCR datasets is reported.

	Methods	Average accuracy			
		Origin	w/ APTF	Win	Tie
128 UCR datasets	FCNet	79.82±0.011	81.21±0.014	82	22
	ResNet	79.93±0.0141	81.04±0.0142	86	25
	InceptionTime	85.90±0.008	86.73±0.008	74	34
	OSCNN	84.82±0.008	85.37±0.0081	65	40
	FormerTime	74.42±0.0157	75.31±0.0152	69	33
	Average	80.97±0.0135	81.93±0.0133	75	31

4.2.3 Time series classification. Our method is not only effective in TSF task but also improves the accuracy of baseline models in TSC task. As shown in Table 3. After applying our APTF, the baselines (FCNet, ResNet, InceptionTime, OSCNN, and FormerTime) achieved higher accuracy on an average of accuracy of 128 UCR datasets. This indicates APTF is a general training framework for both TSF and TSC tasks.

4.3 Ablation study

4.3.1 Effectiveness of hierarchical predictability-aware loss (HPL). HPL incorporates the previous bucket partitioning strategies into the loss calculation and averages the losses obtained from diverse bucket groups. As shown in Figure 2(c) and algorithm 2. This not only helps prevent large changes in gradient direction that can cause training oscillations but also compensates the gradients for highly predictable samples, avoiding the risk that they are not fully learned by the model.

Here we remove the HPL design and use the predictability-aware loss with predictability evolution (Figure 2(b)) for optimizing the model. The result of Table 4 shows that using HPL can reduce forecasting errors by 2.8%, 3.8%, 1.7%, and 2.9% for baselines Autoformer, Scaleformer, NSformer, and NLinear.

Table 4: Ablation study on fund sales datasets. The WMAPE of Fund1, Fund2, and Fund3 on each prediction step are averaged and reported. “w/o Amortize” denotes not using the amortization model and “w/o HPL” denotes removing hierarchical buckets in Figure 2(c) and the design of Figure 2(b) is adopted. The best results are in bold.

		Autoformer-Ours	w/o Amortize	w/o HPL	Scaleformer-Ours	w/o amortize	w/o HPL	NSformer-Ours	w/o Amortize	w/o HPL	NLinear-Ours	w/o Amortize	w/o HPL
Fund-123	1	87.15	88.22	88.29	90.63	92.7	93.19	84.99	86.05	86.22	89.44	91.15	92.55
	5	104.42	107.31	108.33	98.12	100.09	103.04	94.94	96.31	96.96	92.11	92.96	94.37
	8	105.33	106.13	107.03	100.67	102.26	103.16	94.65	96.84	95.92	97.14	97.67	99.09
	10	106.31	108.04	107.88	101.29	103.58	102.3	95.77	96.59	96.38	99.8	100.51	101.62

Table 5: Further demonstrating the effectiveness of hierarchical predictability-aware loss (HPL) on two large-scale datasets. We report the standard deviations and average of MSE and MAE across 96, 192, 336, and 720 prediction steps.

	TimeMixer	PatchTST	Autoformer	NSformer	Scaleformer	NLinear	NHits
Electricity (w/o HPL)	0.306 ±4e-3	0.252 ±3e-4	0.354 ±0.01128	0.338 ±0.01135	0.356 ±0.017	0.364 ±0.002	0.261 ±2e-4
Electricity (w/ HPL)	0.288 ±3e-3	0.249 ±2e-4	0.339 ±0.010	0.325 ±0.007	0.346 ±0.014	0.319 ±0.001	0.254 ±2e-4
Traffic (w/o HPL)	0.655 ±0.013	0.512 ±1e-3	0.654 ±0.02	0.66 ±0.012	0.657 ±0.017	0.794 ±3e-3	0.510 ±6e-3
Traffic (w/ HPL)	0.613 ±0.01	0.502 ±1e-4	0.618 ±0.018	0.634 ±0.011	0.64 ±0.02	0.70 ±3e-3	0.487 ±1e-3

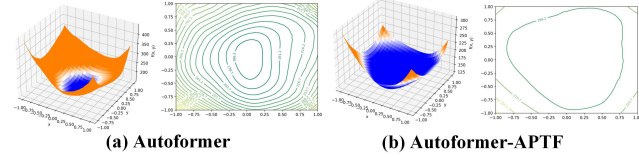


Figure 3: Loss landscapes of Autoformer with and without APTF on the Fund sales dataset for 10-step forecasting. A flatter minimum (larger blue area) indicates better generalization. Additional cases are provided in Appendix Figure 7.

Moreover, we further conduct an ablation study of HPL on two large-scale datasets electricity and traffic, as shown in Table 5. Results indicate that HPL can significantly improve training performance, which further demonstrates the effectiveness of HPL.

4.3.2 Effectiveness of using amortization model. As shown in Table 4, compared to not using an amortization model, using it can reduce forecasting errors by 2.13%, 2.11%, 1.85%, and 1.44% for baselines Autoformer, Scaleformer, NSformer, and NLinear. This indicates that the amortization model can indeed help the source model amortize the predictability estimation errors caused by model bias, thereby further enhancing the performance of HPL.

4.3.3 Benefits on model generalization. It is known that the flatter the loss landscapes [22] of the model, the better the robustness and generalization [28]. Figure 3 shows that the loss landscape of Autoformer becomes significantly flatter after applying APTF, indicating that our framework can indeed improve the model’s robustness and generalization by appropriately penalizing the low-predictability samples during training.

Table 6: Comparison with WaveBound and Co-teaching. We report average performance across all prediction steps (average WMAPE for Fund and average MSE and MAE for other datasets). Hyperparameter search results for Co-teaching are provided in Appendix Table 11.

	ETTh1	ETTm1	weather	Fund1	Fund2	Fund3
TimeMixer-Original	0.517	0.404	0.269	<u>89.963</u>	<u>85.512</u>	<u>85.14</u>
-Wavebound	0.706	0.489	0.279	206.9	186.198	328.27
-Coteaching	<u>0.508</u>	<u>0.402</u>	<u>0.268</u>	100.23	95.716	94.893
-APTF (Ours)	0.492	0.396	0.264	87.91	84.36	83.659
PatchTST-Original	0.430	0.369	0.244	88.811	<u>86.472</u>	<u>86.593</u>
-Wavebound	0.434	<u>0.364</u>	0.245	191.085	197.374	201.886
-Coteaching	<u>0.429</u>	0.37	0.245	<u>88.604</u>	88.668	87.022
-APTF (Ours)	0.420	0.357	0.242	86.55	85.09	84.821
NSformer-Original	0.520	0.599	0.312	97.199	<u>94.406</u>	<u>92.646</u>
-Wavebound	0.528	0.595	<u>0.309</u>	193.569	181.116	211.739
-Coteaching	<u>0.516</u>	<u>0.582</u>	0.312	<u>96.744</u>	94.847	93.348
-APTF (Ours)	0.511	0.553	0.307	95.364	91.826	90.579

4.3.4 Compared with framework Co-teaching and Wavebound. The comparison results between APTF and baseline training frameworks Wavebound [9] and Co-teaching [14] are shown in Table 6. The results show that APTF outperforms both of them, especially for the fund sales dataset that suffers from high noise due to complex factors such as unexpected policy and public opinion. The advantage of our method lies in imposing appropriate penalties on the losses of samples, mitigating the adverse effects caused by excessively large losses of low-predictability samples, and enabling the model to better learn from both highly predictable and low-predictability samples. In contrast, Co-teaching’s approach of directly discarding samples with high loss values may lead to knowledge waste and harm model performance. Additionally, when the error falls below a certain threshold, Wavebound collectively applies gradient ascent to all samples, which may impact the model’s ability to learn from some highly predictable samples.

4.3.5 Compared with predictability-aware loss functions. Table 7 shows that our APTF achieves better overall performance than adaptive robust loss (ARL) and Self-paced learning (SPL). In addition, we also empirically observe that APTF significantly outperforms ARL and SPL on three Fund datasets that suffer from high noise, further demonstrating its effectiveness. Overall, the experiment results demonstrate that our APTF can better address low-predictability samples to improve model performance. In contrast, SPL directly discards low-predictability samples in the early

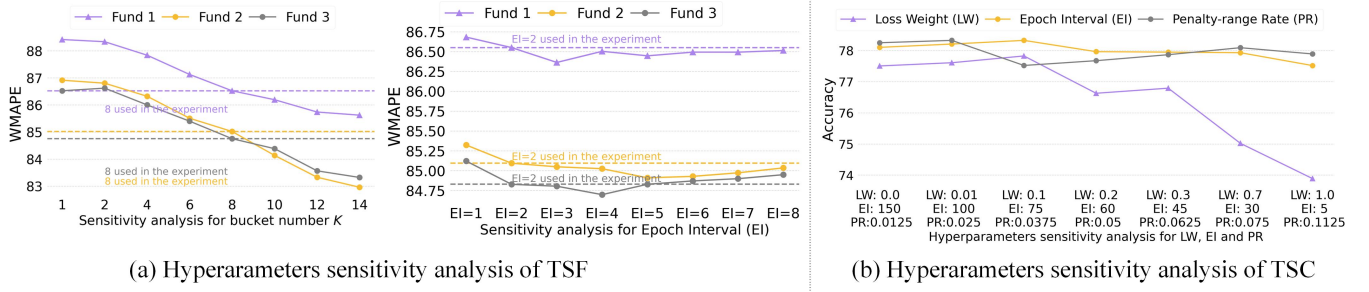


Figure 4: Hyperparameter sensitivity analysis.

stages, similar to Co-teaching, which may lead to underfitting as the model fails to learn sufficiently from the knowledge contained in low-predictability samples. The poor performance of ARL indicates that relying solely on adaptive parameters may not impose sufficient constraints on low-predictability samples, thereby impairing model convergence.

Table 7: Comparison with predictability-aware loss functions on the Electricity dataset. We report the average MSE and MAE over prediction horizons of 96, 192, 336, and 720 steps. ARL [3] denotes adaptive robust loss, while SPL [21] denotes self-paced learning loss.

	TimeMixer	Autoformer	NSFormer	Scaleformer	NLinear	NHits
Original	0.306	0.344	0.330	0.35	0.355	0.262
+APTF	0.288	0.335	0.321	0.336	0.319	0.255
+ARL	0.307	0.357	0.329	0.347	0.367	0.273
+SPL	0.300	0.349	0.332	0.451	0.360	0.262

4.4 Time complexity and memory cost analysis

When using the amortized model, our method has approximately double the memory overhead and time complexity of the original model. Without amortizing, there are almost no additional costs. For instance, on the weather dataset, the average training epoch time is **51.75s VS 52.99s** for PatchTST VS Ours (PatchTST-APTF-w/o amortization), **52.89s VS 53.20s** for Scaleformer VS Ours, and **40.07s VS 42.75s** for Autoformer VS Ours. More importantly, our method still brings significant accuracy improvements to the baseline model without using the amortized model, as shown in Tables 2 and Table 8. Therefore, **our method can improve model performance without obviously increasing model overhead when not using the amortization model**, which further demonstrates the effectiveness of our approach.

4.5 Hyperparameter sensitivity analysis

We analyze hyperparameters such as the number of buckets and epoch intervals. As shown in Figure 4, for TSF, we observe that the prediction error continuously decreases with the increasing number of buckets in each hierarchical bucket group. This is reasonable because more buckets result in a finer bucket division and the low-predictability samples will be penalized more accurately, thereby improving model performance. When the number of buckets exceeds 15, the improvement in model performance tends to

Table 8: APTF can still improve baseline accuracy without introducing additional time complexity or memory overhead when the amortization model is not used. We report the average WMAPE for the Fund datasets and the average MSE for the other datasets.

	ET.h1	ET.h2	ET.m1	ET.m2	Fund1	Fund2	Fund3
TimeMixer-Original	0.537	0.405	0.402	0.281	89.96	85.51	85.14
-Ours (w/o amortization)	0.512	0.385	0.396	0.279	88.5	84.62	84.02
PatchTST-Original	0.423	0.347	0.351	0.258	88.81	86.47	86.59
-Ours (w/o amortization)	0.416	0.345	0.345	0.250	87.2	85.7	85.45
NHiTS-Original	0.526	0.612	0.386	0.307	91.69	91.53	90.33
-Ours (w/o amortization)	0.499	0.582	0.373	0.291	87.9	87.1	85.8
NSFormer-Original	0.539	0.419	0.657	0.335	97.2	94.41	92.65
-Ours (w/o amortization)	0.535	0.396	0.618	0.318	95.97	92.34	91.73

saturate. It is worth noting that the maximum number of buckets in the experiments is set to 9, which is not optimal. We also observe that the better epoch interval ϵ is 3 or 4, while used in the paper is 2, which is not optimal. For TSC, we observe that the loss weight, epoch interval, and penalty range rate are set as 0.1, 75, and 0.025, respectively can achieve overall good performance.

5 Conclusion

This paper proposes the Amortized Predictability-aware Training Framework (APTF) to enhance the training of baseline models in both TSF and TSC tasks. APTF boosts model convergence through two key designs: the Hierarchical Predictability-aware Loss (HPL) and the amortization model. These components enable the model to focus on high-predictability samples while still learning appropriately from low-predictability ones, with a reduced risk of overfitting to noise data, thereby improving overall training performance. Extensive experiments on popular TSF and TSC datasets demonstrate that APTF consistently enhances the convergence of diverse advanced models, including eleven TSF and five TSC baselines.

6 Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant 62427819.

References

- [1] Devansh Arpit, Stanislaw Jastrzębski, Nicolas Ballas, David Krueger, Emmanuel Bengio, Maxinder S Kanwal, Tegan Maharaj, Asja Fischer, Aaron Courville, Yoshua Bengio, et al. 2017. A closer look at memorization in deep networks. In *International conference on machine learning*. PMLR, 233–242.
- [2] Maria-Florina Balcan, Alina Beygelzimer, and John Langford. 2006. Agnostic active learning. In *Proceedings of the 23rd international conference on Machine learning*. 65–72.
- [3] Jonathan T Barron. 2019. A general and adaptive robust loss function. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 4331–4339.
- [4] Avrim Blum and Tom Mitchell. 1998. Combining labeled and unlabeled data with co-training. In *Proceedings of the eleventh annual conference on Computational learning theory*. 92–100.
- [5] C Challu, KG Olivares, BN Oreshkin, F Garza, M Mergenthaler, and A Dubrawski. 2022. N-hits: Neural hierarchical interpolation for time series forecasting. *arXiv preprint arXiv:2201.12886* (2022).
- [6] Si-An Chen, Chun-Liang Li, Nate Yoder, Serkan O Arik, and Tomas Pfister. 2023. Tsmixer: An all-mlp architecture for time series forecasting. *arXiv preprint arXiv:2303.06053* (2023).
- [7] Mingyue Cheng, Qi Liu, Zhiding Liu, Zhi Li, Yucong Luo, and Enhong Chen. 2023. Formertime: Hierarchical multi-scale representations for multivariate time series classification. In *Proceedings of the ACM Web Conference 2023*. 1437–1445.
- [8] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078* (2014).
- [9] Youngjin Cho, Daejin Kim, Dongmin Kim, Mohammad Azam Khan, and Jaegul Choo. 2022. WaveBound: dynamic error bounds for stable time series forecasting. *Advances in Neural Information Processing Systems* 35 (2022), 19579–19591.
- [10] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. 2023. Long-term Forecasting with TiDE: Time-series Dense Encoder. *Trans. Mach. Learn. Res.* 2023 (2023). <https://openreview.net/forum?id=pCbC3aQB5W>
- [11] Francis X Diebold and Georg Strasser. 2013. On the correlation structure of microstructure noise: A financial economic approach. *Review of Economic Studies* 80, 4 (2013), 1304–1337.
- [12] Germain Forestier, François Petitjean, Pavel Senin, Fabien Despinoy, Arnaud Huau, Hassan Ismail Fawaz, Jonathan Weber, Lhassane Idoumghar, Pierre-Alain Muller, and Pierre Jannin. 2018. Surgical motion analysis using discriminative interpretable patterns. *Artificial intelligence in medicine* 91 (2018), 3–11.
- [13] Yoav Freund, Robert Schapire, and Naoki Abe. 1999. A short introduction to boosting. *Journal-Japanese Society For Artificial Intelligence* 14, 771-780 (1999), 1612.
- [14] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. 2018. Co-teaching: Robust training of deep neural networks with extremely noisy labels. *Advances in neural information processing systems* 31 (2018).
- [15] Tao Huang, Pengfei Chen, and Ruipeng Li. 2022. A semi-supervised vae based active anomaly detection framework in multivariate time series for online systems. In *Proceedings of the ACM Web Conference 2022*. 1797–1806.
- [16] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. 2019. Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery* (Jul 2019), 917–963. doi:10.1007/s10618-019-00619-1
- [17] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F Schmidt, Jonathan Weber, Geoffrey I Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. 2020. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery* 34, 6 (2020), 1936–1962.
- [18] Lu Jiang, Zhengyuan Zhou, Thomas Leung, Li-Jia Li, and Li Fei-Fei. 2018. Mentor-net: Learning data-driven curriculum for very deep neural networks on corrupted labels. In *International conference on machine learning*. PMLR, 2304–2313.
- [19] Michael I Jordan. 1997. Serial order: A parallel distributed processing approach. In *Advances in psychology*. Vol. 121. Elsevier, 471–495.
- [20] D Kim, J Park, J Lee, and H Kim. 2024. Are Self-Attentions Effective for Time Series Forecasting?. In *38th Conference on Neural Information Processing Systems (NeurIPS 2024)*.
- [21] M Kumar, Benjamin Packer, and Daphne Koller. 2010. Self-paced learning for latent variable models. *Advances in neural information processing systems* 23 (2010).
- [22] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2018. Visualizing the loss landscape of neural nets. *Advances in neural information processing systems* 31 (2018).
- [23] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Non-stationary transformers: Exploring the stationarity in time series forecasting. *Advances in Neural Information Processing Systems* 35 (2022), 9881–9893.
- [24] Donghao Luo and Xue Wang. 2024. Modernrtn: A modern pure convolution structure for general time series analysis. In *The twelfth international conference on learning representations*. 1–43.
- [25] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. [n. d.]. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations*.
- [26] P Nunes, J Santos, and E Rocha. 2023. Challenges in predictive maintenance—A review. *CIRP Journal of Manufacturing Science and Technology* 40 (2023), 53–67.
- [27] Diptangshu Pandit, Li Zhang, Chengyu Liu, Nauman Aslam, Samiran Chattopadhyay, and Chee Peng Lim. 2017. Noise reduction in ECG signals using wavelet transform and dynamic thresholding. *Emerging trends in neuro engineering and neural computation* (2017), 193–206.
- [28] Namuk Park and Songkuk Kim. 2022. How Do Vision Transformers Work?. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. <https://openreview.net/forum?id=D78Go4hVcxO>
- [29] Pritam Sarkar and Ali Etemad. 2020. Self-supervised ECG representation learning for emotion recognition. *IEEE Transactions on Affective Computing* 13, 3 (2020), 1541–1554.
- [30] Rajat Sen, Hsiang-Fu Yu, and Inderjit S Dhillon. 2019. Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting. *Advances in neural information processing systems* 32 (2019).
- [31] Amin Shabani, Amir Abdi, Lili Meng, and Tristan Sylvain. 2022. Scaleformer: iterative multi-scale refining transformers for time series forecasting. *arXiv preprint arXiv:2206.04038* (2022).
- [32] Wensi Tang, Guodong Long, Lu Liu, Tianyi Zhou, Michael Blumenstein, and Jing Jiang. 2022. Omni-Scale CNNs: a simple and effective kernel size configuration for time series classification. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. <https://openreview.net/forum?id=PDYs7Z2XFGv>
- [33] Wensi Tang, Guodong Long, Lu Liu, Tianyi Zhou, Jing Jiang, and Michael Blumenstein. 2020. Rethinking 1d-cnn for time series classification: A stronger baseline. *arXiv preprint arXiv:2002.10061* (2020), 1–7.
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [35] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y Zhang, and Jun Zhou. 2024. TimeMixer: Decomposable Multiscale Mixing for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*.
- [36] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long. [n. d.]. TimeXer: Empowering Transformers for Time Series Forecasting with Exogenous Variables. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [37] Zhiguang Wang, Weizhong Yan, and Tim Oates. 2017. Time series classification from scratch with deep neural networks: A strong baseline. In *2017 International joint conference on neural networks (IJCNN)*. IEEE, 1578–1585.
- [38] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in Neural Information Processing Systems* 34 (2021), 22419–22430.
- [39] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.
- [40] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. 2021. Understanding deep learning (still) requires rethinking generalization. *Commun. ACM* 64, 3 (2021), 107–115.
- [41] Xu Zhang, Zhengang Huang, Yunzhi Wu, Xun Lu, Erpeng Qi, Yunkai Chen, Zhongya Xue, Peng Wang, and Wei Wang. 2024. Self-Adaptive Scale Handling for Forecasting Time Series with Scale Heterogeneity. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 7485–7489.
- [42] Xu Zhang, Zhengang Huang, Yunzhi Wu, Xun Lu, Erpeng Qi, Yunkai Chen, Zhongya Xue, Qitong Wang, Peng Wang, and Wei Wang. 2025. Multi-period Learning for Financial Time Series Forecasting. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2848–2859.
- [43] Xu Zhang, Peng Wang, Chen Wang, Zhe Xu, Xiaohua Nie, and Wei Wang. 2025. Global Feature Enhancing and Fusion Framework for Strain Gauge Status Recognition. In *Companion Proceedings of the ACM on Web Conference 2025*. 611–620.
- [44] Xu Zhang, Qitong Wang, Peng Wang, and Wei Wang. 2025. A Lightweight Sparse Interaction Network for Time Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 13304–13312.
- [45] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.
- [46] Tian Zhou, Ziqing Ma, Qingsong Wen, Liang Sun, Tao Yao, Wotao Yin, Rong Jin, et al. 2022. Film: Frequency improved legendre memory model for long-term time series forecasting. *Advances in Neural Information Processing Systems* 35 (2022), 12677–12690.

A Appendices

A.1 Experimental settings

A.1.1 Implementation details. Time Series Forecasting (TSF). Training uses a fixed epoch interval $\varepsilon = 2$ and 30 epochs for all methods, with input lengths of 96 or 512. The initial number of buckets is 9 and decreases by 1 at each training stage. Loss weights in the first stage are uniformly decreasing: the first bucket has weight 1, the last is half of the previous bucket, and intermediate buckets decrease evenly with interval $1/(K - 1)$, where K is the number of buckets. In subsequent stages, the largest weight is removed. Time Series Classification (TSC). Each batch is split into high- and low-predictability buckets with loss weights 1 and $\in [0.1, 0.01]$, respectively. Training runs for 300 epochs with $\varepsilon = 75$, gradually increasing the proportion of low-predictability samples by 0.025 per stage to expand gradient penalties.

A.1.2 Discussion and visualization of Fund sales datasets (Figure 5). Fund sales time series exhibit sharp trends and frequent fluctuations due to market conditions, policies, and public opinion (Figure 5), leading to many low-predictability samples that hinder training. This makes the fund dataset well suited for evaluating our framework’s ability to identify and penalize such samples, where improved accuracy indicates better handling of unpredictability and enhanced convergence.

A.2 Additional experiment results

A.2.1 More baselines (CATS, ModernTCN and TimeXer) on short-term forecasting (Table 9) and long-term forecasting (Table 10). Here, we further demonstrate the application of APTF to CATS, ModernTCN, and TimeXer for both short-term and long-term forecasting. The results in Table 9 and Table 10) show that APTF consistently and significantly improves the training performance of these baselines, further proving that APTF is generalizable, universal, and a model-agnostic framework.

A.2.2 Hyperparameter search of Co-teaching for fair comparison (Table 11). As shown in Table 11, we have conducted a hyperparameter search for Co-teaching, and its best performance is still inferior to ours, compared with Table 6 of the manuscript. This indicates that the approach of directly discarding high-loss samples in Co-teaching indeed has limitations, while our method effectively addresses this issue and achieves better performance.

A.2.3 Visualization of forecasting (Figure 6). We further provide qualitative visualizations to illustrate APTF’s effectiveness. In Figure 6, models trained with APTF better capture overall trends, indicating that dynamically penalizing low-predictability samples and emphasizing high-predictability ones improves performance.

A.2.4 Full results on model generalization (Figure 7). We also present more experimental results on whether APTF can bring generalizability to models, as shown in Figure 7. The results indicate that training based on the APTF framework tends to make models converge to flatter regions of the loss landscape, thereby achieving better generalizability.

A.2.5 Results of long-term forecasting on each prediction length (Table 12 and Table 13). The results show that APTF consistently improves the performance of baseline models across

different prediction lengths, which illustrates the effectiveness and generalizability of the APTF framework.

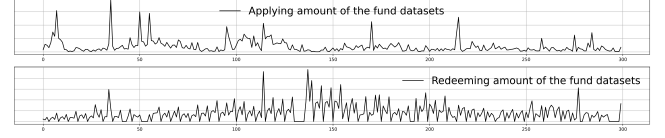


Figure 5: Time series visualization of Fund sales dataset.

Table 9: Additional baselines for short-term TSF with average WMAPE on 1, 5, 8, and 10 future steps. Results for each prediction length are omitted due to limited space.

	CATS		ModernTCN		TimeXer	
	+APTF	Original	+APTF	Original	+APTF	Original
Fund1	85.72	88.17	86.47	90.28	83.97	86.29
Fund2	82.92	84.79	83.83	85.07	82.1	83.55
Fund3	80.56	83.91	82.55	85.16	81.38	84.02

Table 10: Additional baselines for long-term TSF with average MSE and MAE on future 96, 192, 336, and 720 steps. Table 12 shows results for each prediction length.

	CATS		ModernTCN		TimeXer	
	+APTF	Original	+APTF	Original	+APTF	Original
ETTh1	0.426	0.430	0.426	0.438	0.436	0.445
ETTh2	0.363	0.373	0.365	0.374	0.367	0.373
ETTm1	0.357	0.366	0.371	0.386	0.377	0.382
ETTm2	0.288	0.313	0.289	0.302	0.284	0.295
Weather	0.245	0.246	0.246	0.251	0.248	0.249
Exchange	0.364	0.376	0.369	0.38	0.378	0.388
Electricity	0.270	0.275	0.235	0.243	0.284	0.289
Traffic	0.611	0.627	0.509	0.529	0.558	0.564

Table 11: Hyperparameter search for Coteaching. We report the average of MSE and MAE on predicting all timesteps. The forget rate r determines the number of samples discarded during the training process.

Forget rate r	ETTh1	ETTm1	Weather	Fund1	Fund2	Fund3
TimeMixer, $r=0.1$	0.509	0.403	0.268	100.23	95.716	94.893
TimeMixer, $r=0.2$	0.508	0.403	0.269	101.29	96.11	95.571
TimeMixer, $r=0.3$	0.508	0.402	0.27	102.278	96.846	96.057
TimeMixer, $r=0.4$	0.509	0.402	0.27	102.972	97.133	96.714
PatchTST, $r=0.1$	0.429	0.37	0.245	88.604	88.668	87.022
PatchTST, $r=0.2$	0.429	0.371	0.246	89.706	90.582	88.074
PatchTST, $r=0.3$	0.429	0.372	0.246	91.421	94.287	89.612
PatchTST, $r=0.4$	0.43	0.372	0.246	92.81	97.306	92.284
NSFormer, $r=0.1$	0.516	0.584	0.312	96.744	94.847	93.348
NSFormer, $r=0.2$	0.518	0.586	0.312	97.481	95.427	93.664
NSFormer, $r=0.3$	0.519	0.582	0.314	98.354	96.752	94.328
NSFormer, $r=0.4$	0.517	0.582	0.314	98.932	97.242	95.423

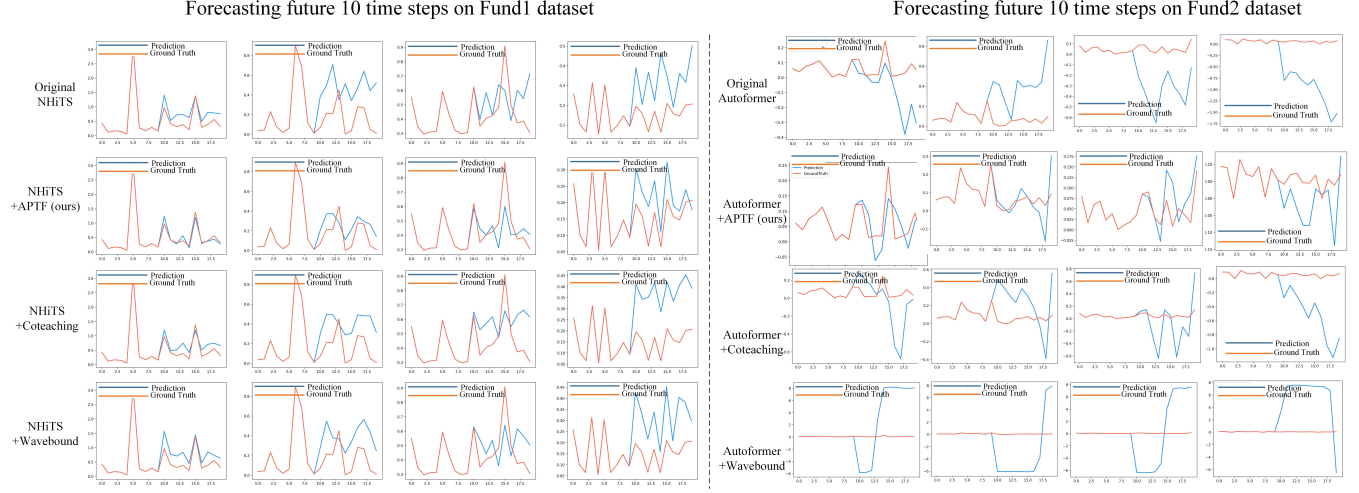


Figure 6: Visualizations of forecasting future 10-time steps on Fund1 and Fund2 datasets with the original baseline NHITS (left) and Autoformer (right), as well as the versions of applying APTF (ours), Coteaching, and Wavebound.

Table 12: Full results of CATS, ModernTCN, and TimeXer on long-term TSF on popular public datasets with average performance on four random seeds. Due to space limitations, the standard deviation is omitted, and we will show it in our full version paper.

		CATS-APTF		CATS		ModernTCN-APTF		ModernTCN		TimeXer-APTF		TimeXer	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.379	0.4	0.389	0.408	0.37	0.399	0.381	0.411	0.394	0.412	0.396	0.416
	192	0.4	0.417	0.409	0.424	0.403	0.422	0.42	0.439	0.421	0.431	0.423	0.438
	336	0.421	0.434	0.422	0.436	0.428	0.438	0.441	0.452	0.436	0.448	0.441	0.459
	720	0.482	0.473	0.482	0.471	0.47	0.477	0.476	0.481	0.462	0.485	0.488	0.498
ETTh2	96	0.278	0.345	0.28	0.35	0.269	0.335	0.279	0.347	0.278	0.346	0.279	0.347
	192	0.331	0.38	0.334	0.389	0.33	0.374	0.333	0.384	0.34	0.383	0.348	0.39
	336	0.354	0.402	0.36	0.41	0.365	0.407	0.369	0.412	0.363	0.405	0.366	0.41
	720	0.386	0.428	0.413	0.448	0.403	0.435	0.419	0.449	0.392	0.429	0.406	0.443
ETTm1	96	0.29	0.34	0.299	0.352	0.298	0.347	0.308	0.36	0.309	0.357	0.315	0.363
	192	0.319	0.359	0.327	0.368	0.346	0.375	0.354	0.382	0.35	0.38	0.353	0.385
	336	0.351	0.378	0.358	0.387	0.374	0.388	0.393	0.405	0.382	0.4	0.382	0.404
	720	0.406	0.413	0.418	0.423	0.418	0.421	0.446	0.44	0.414	0.425	0.425	0.433
ETTm2	96	0.165	0.254	0.189	0.283	0.161	0.249	0.169	0.26	0.165	0.254	0.174	0.262
	192	0.219	0.293	0.245	0.321	0.217	0.288	0.226	0.301	0.222	0.294	0.236	0.306
	336	0.274	0.33	0.302	0.356	0.28	0.329	0.292	0.342	0.272	0.326	0.287	0.338
	720	0.374	0.398	0.393	0.413	0.392	0.397	0.414	0.412	0.357	0.38	0.367	0.391
Weather	96	0.147	0.202	0.148	0.205	0.142	0.191	0.151	0.205	0.152	0.204	0.156	0.209
	192	0.191	0.244	0.192	0.246	0.192	0.241	0.196	0.248	0.199	0.246	0.199	0.247
	336	0.242	0.283	0.243	0.285	0.247	0.285	0.249	0.288	0.248	0.284	0.249	0.285
	720	0.316	0.336	0.316	0.336	0.328	0.34	0.331	0.342	0.314	0.333	0.313	0.334
Exchange	96	0.083	0.202	0.084	0.203	0.082	0.199	0.085	0.203	0.087	0.207	0.087	0.208
	192	0.17	0.295	0.175	0.298	0.168	0.292	0.173	0.296	0.179	0.302	0.181	0.302
	336	0.314	0.404	0.326	0.412	0.308	0.402	0.325	0.412	0.326	0.414	0.338	0.42
	720	0.78	0.663	0.825	0.684	0.823	0.682	0.854	0.698	0.825	0.685	0.865	0.702
Electricity	96	0.202	0.28	0.209	0.287	0.152	0.255	0.157	0.263	0.216	0.315	0.221	0.32
	192	0.215	0.292	0.22	0.297	0.171	0.272	0.181	0.284	0.219	0.318	0.224	0.323
	336	0.233	0.308	0.236	0.312	0.189	0.291	0.198	0.3	0.234	0.332	0.24	0.337
	720	0.281	0.349	0.286	0.355	0.226	0.323	0.234	0.33	0.277	0.363	0.282	0.368
Traffic	96	0.758	0.456	0.767	0.465	0.562	0.348	0.579	0.357	0.698	0.439	0.703	0.442
	192	0.725	0.445	0.741	0.457	0.596	0.374	0.621	0.388	0.662	0.426	0.669	0.43
	336	0.734	0.448	0.751	0.46	0.633	0.396	0.665	0.413	0.665	0.422	0.678	0.432
	720	0.824	0.496	0.855	0.519	0.726	0.437	0.758	0.45	0.707	0.441	0.714	0.445

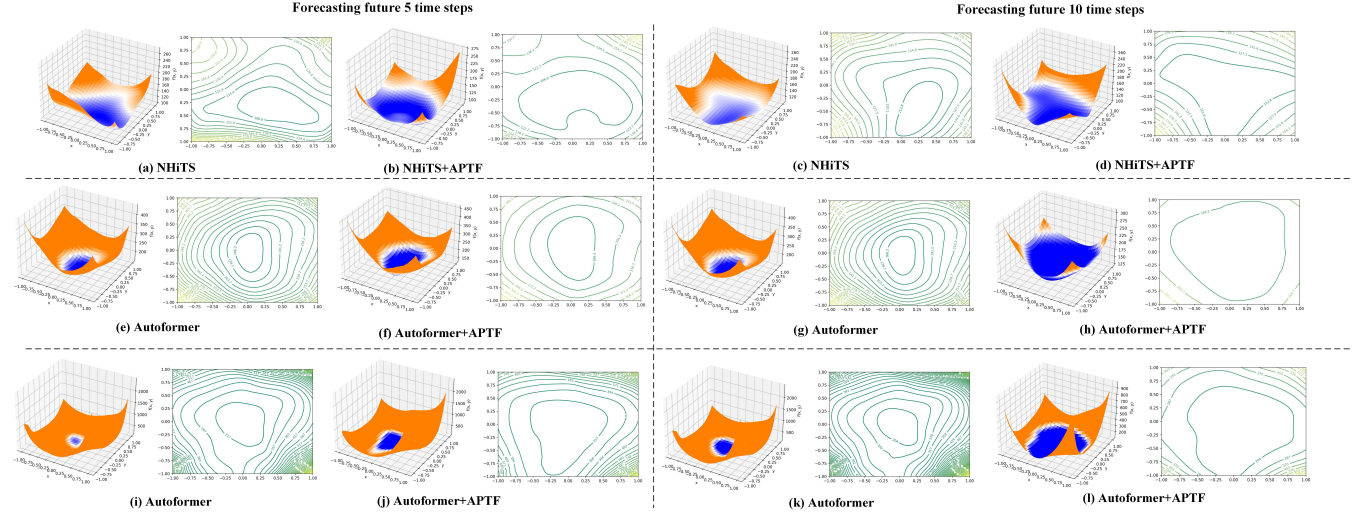


Figure 7: Full results of loss landscapes on different baselines with the Fund1 dataset. The flatter the bottom (the larger the blue area) indicates the better the generalization of the trained models.

Table 13: Full results of long-term forecasting on remaining baselines. with average performance on four random seeds. Due to space limitations, the standard deviation is omitted, and we will show it in our full version paper.

		TimeMixer		PatchTST		Autoformer		NSformer		Scaleformer		NLinear		N-HiTS		Informer	
		+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original	+APTF	Original
ETTh1	96	0.434	0.45	0.377	0.38	0.465	0.492	0.512	0.544	0.506	0.507	0.423	0.442	0.432	0.448	0.9	0.948
	192	0.49	0.522	0.406	0.412	0.479	0.494	0.5	0.51	0.494	0.495	0.449	0.464	0.466	0.484	1.031	1.05
	336	0.512	0.535	0.427	0.436	0.476	0.484	0.49	0.496	0.566	0.579	0.466	0.478	0.496	0.516	1.079	1.056
	720	0.532	0.56	0.472	0.49	0.548	0.544	0.541	0.542	0.607	0.619	0.475	0.492	0.57	0.616	1.141	1.176
ETTh2	96	0.316	0.326	0.304	0.31	0.376	0.409	0.39	0.406	0.381	0.408	0.318	0.328	0.349	0.385	1.225	1.568
	192	0.396	0.43	0.357	0.369	0.382	0.412	0.396	0.418	0.39	0.429	0.372	0.378	0.416	0.470	1.471	2.579
	336	0.43	0.446	0.388	0.386	0.386	0.422	0.409	0.428	0.4	0.436	0.403	0.401	0.568	0.631	1.589	2.718
	720	0.44	0.457	0.414	0.415	0.446	0.494	0.434	0.482	0.464	0.492	0.426	0.426	0.853	0.91	2.024	2.562
ETTm1	96	0.343	0.356	0.308	0.316	0.489	0.502	0.519	0.526	0.484	0.478	0.334	0.334	0.334	0.348	0.514	0.652
	192	0.385	0.396	0.342	0.355	0.526	0.558	0.535	0.566	0.514	0.542	0.36	0.362	0.371	0.382	0.613	0.707
	336	0.407	0.414	0.369	0.387	0.525	0.605	0.571	0.649	0.521	0.528	0.382	0.386	0.394	0.41	0.712	0.75
	720	0.456	0.461	0.41	0.419	0.485	0.559	0.588	0.656	0.504	0.526	0.423	0.428	0.44	0.456	0.861	0.852
ETTm2	96	0.216	0.221	0.202	0.212	0.31	0.318	0.282	0.302	0.278	0.293	0.21	0.217	0.224	0.244	0.888	1.163
	192	0.27	0.274	0.249	0.264	0.332	0.338	0.316	0.33	0.343	0.391	0.256	0.262	0.277	0.298	1.048	1.204
	336	0.318	0.323	0.294	0.31	0.353	0.362	0.344	0.361	0.367	0.463	0.302	0.306	0.334	0.356	1.692	2.142
	720	0.395	0.398	0.366	0.375	0.408	0.424	0.408	0.432	0.428	0.508	0.374	0.378	0.408	0.44	2.572	3.857
weather	96	0.192	0.196	0.169	0.173	0.351	0.357	0.246	0.272	0.326	0.372	0.198	0.207	0.182	0.188	0.316	0.337
	192	0.235	0.24	0.213	0.216	0.37	0.37	0.282	0.288	0.359	0.58	0.238	0.246	0.229	0.236	0.43	0.436
	336	0.284	0.29	0.262	0.263	0.39	0.397	0.319	0.32	0.498	0.559	0.279	0.286	0.278	0.282	0.53	0.58
	720	0.346	0.35	0.325	0.324	0.42	0.418	0.382	0.374	0.627	0.623	0.338	0.34	0.34	0.344	0.756	1.027
Exchange	96	0.146	0.15	0.142	0.143	0.206	0.218	0.2	0.211	0.2	0.213	0.144	0.147	0.168	0.172	0.582	0.706
	192	0.237	0.242	0.23	0.233	0.306	0.32	0.308	0.308	0.302	0.324	0.232	0.236	0.251	0.309	0.816	0.88
	336	0.359	0.368	0.355	0.362	0.446	0.459	0.436	0.442	0.455	0.518	0.357	0.364	0.386	0.504	1.002	1.006
	720	0.742	0.764	0.741	0.756	0.863	0.956	0.831	0.881	1.01	1.192	0.746	0.756	0.619	0.622	1.099	1.102
Electricity	96	0.262	0.279	0.23	0.232	0.304	0.324	0.282	0.29	0.288	0.308	0.302	0.338	0.232	0.238	0.588	0.596
	192	0.272	0.291	0.234	0.237	0.327	0.328	0.308	0.32	0.318	0.324	0.304	0.341	0.239	0.246	0.708	0.75
	336	0.29	0.308	0.249	0.252	0.334	0.34	0.331	0.334	0.348	0.37	0.318	0.355	0.256	0.264	0.804	0.821
	720	0.328	0.344	0.286	0.289	0.376	0.382	0.363	0.378	0.39	0.396	0.352	0.387	0.292	0.299	0.85	0.854
Traffic	96	0.619	0.648	0.512	0.514	0.59	0.615	0.568	0.588	0.563	0.609	0.714	0.799	0.472	0.482	0.736	0.74
	192	0.594	0.623	0.479	0.485	0.64	0.642	0.659	0.694	0.671	0.656	0.681	0.778	0.472	0.486	0.902	0.913
	336	0.594	0.633	0.486	0.49	0.604	0.638	0.64	0.668	0.657	0.692	0.689	0.788	0.486	0.501	1.039	1.112
	720	0.644	0.687	0.515	0.517	0.637	0.674	0.663	0.692	0.665	0.702	0.716	0.817	0.526	0.542	1.137	1.138